

NairalQ — 3■Page Summary

NairalQ — Project Summary

Purpose

NairalQ is a modular fintech backend focused on Nigerian users. It provides deterministic financial profiling from onboarding inputs, producing a 12-bit feature vector and a pressure score that informs downstream recommendations.

Tech stack

- FastAPI (ASGI)
- Async SQLAlchemy (PostgreSQL)
- Alembic migrations
- Pydantic v2 + pydantic-settings
- JWT authentication, bcrypt for passwords
- Testing: pytest + pytest-asyncio, httpx for HTTP tests

Architecture

Clean Architecture style: domain features and rules are separated from repositories and API routes. Profiles are versioned (`is_active`) and stored as JSONB for features, vectors and scores.

Changes & Files

Key changes applied (high level)

- Added ``pydantic-settings`` and migrated settings to Pydantic v2 usage.
- Enforced onboarding authentication: ``user_id`` no longer accepted in request body; route uses ``get_current_user``.
- Implemented deterministic feature extractor with ``FEATURE_NAMES`` (12 binary features) and a ``vectorize()`` that returns exactly 12 binary integers.
- Replaced simplistic scoring with ``compute_pressure_score()`` that aggregates rule scores into ``pressure_score``, ``pressure_label``, ``overall`` and ``breakdown``.
- Added profile versioning: ``FinancialProfile.is_active`` boolean column and migration ``0002_add_is_active_to_profiles.py``.
- Repository changes: ``deactivate_prior_profiles(user_id)`` added and ``get_by_user_id()`` filters active profiles.
- Tests: unit tests adjusted to expect 12-feature vectors; integration tests added as endpoint smoke tests.

Tests & Next Steps

Tests & Run notes

- Unit tests (suite): 5 passed.
- Measured in this environment: 5 passed in 3.15s
- User-reported unit test runtime: 0.121s
- Integration checks performed (ASGI):
- POST /api/v1/onboarding/ returned 401 (authentication enforced)
- POST /api/v1/auth/register encountered an external DNS/connection error during checks

Next steps

1. Provide a reachable PostgreSQL `DATABASE_URL` or configure tests to use a local test DB to run integration tests fully.
2. Rotate any secrets if `.env` was committed; add `.env` to `.gitignore` and use `.env.example` as a template.
3. Create a private GitHub repo and push the project; consider secret scanning and protected branches.

Generated: 2026-05-22 09:35:53